

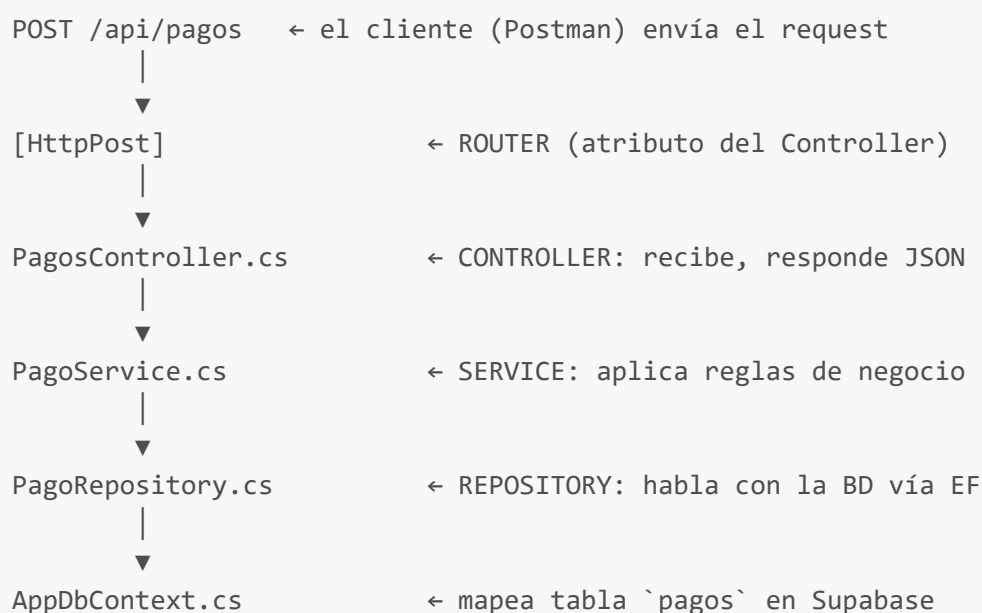
Guía didáctica — API de Pagos de Servicios

Stack: ASP.NET Core 8 · Entity Framework Core · PostgreSQL (Supabase) · VS Code **Patrón:** ROUTER → CONTROLLER → SERVICE → REPOSITORY → BASE DE DATOS

1. ¿Qué vamos a construir?

Una API REST con CRUD completo para gestionar **pagos de servicios** (agua, electricidad, streaming, internet, gas).

Flujo de una petición HTTP



¿Por qué 4 capas?

Cada capa tiene **una sola responsabilidad**. Si mañana cambias de Supabase a SQL Server, solo tocas el Repository. Si cambias una regla de negocio (ej. aceptar otro servicio), solo tocas el Service. El Controller no sabe ni de reglas ni de BD — solo de HTTP.

2. Requisitos previos

2.1. Instalar .NET SDK 8.0

1. Descargar desde dotnet.microsoft.com/download.
2. Verificar en terminal:

```
dotnet --version
```

```
# → 8.0.xxx
```

2.2. Extensiones de VS Code

- **C# Dev Kit** — autocompletado, debugger, explorador de soluciones.
- **C#** — soporte base del lenguaje (se instala automáticamente con Dev Kit).

2.3. Cuenta de Supabase

1. Crear un proyecto gratuito en supabase.com.
2. Ir a **Project Settings** → **Database** → **Connection string** y anotar:
 - Host (ej. `db.xxxxxxx.supabase.co`)
 - Puerto: `5432`
 - Database: `postgres`
 - User: `postgres`
 - Password: el que definiste al crear el proyecto

3. Estructura final del proyecto

```
s2_m4_asp_net_supabase/  
├── Controllers/  
│   └── PagosController.cs      ← CAPA CONTROLLER  
├── Services/  
│   ├── IPagoService.cs        ← Interfaz del Service  
│   └── PagoService.cs         ← CAPA SERVICE  
├── Repositories/  
│   ├── IPagoRepository.cs     ← Interfaz del Repository  
│   └── PagoRepository.cs       ← CAPA REPOSITORY  
├── Models/  
│   └── Pago.cs                 ← Entidad (mapea tabla)  
├── Data/  
│   └── AppDbContext.cs         ← Contexto de EF Core  
├── Migrations/  
├── Program.cs                  ← entrada + DI  
└── appsettings.json            ← configuración de BD
```

4. Crear el proyecto

Abre la terminal de VS Code (**Ctrl** + **`**) en la carpeta donde quieres trabajar:

```
dotnet new webapi -n s2_m4_asp_net_supabase  
cd s2_m4_asp_net_supabase  
code .
```

¿Qué hace **webapi**? Es la plantilla oficial para APIs REST: ya trae **Program.cs**, **appsettings.json**, un controller de ejemplo (**WeatherForecast**) que luego borras, y el archivo **.csproj**.

5. Instalar paquetes NuGet

```
dotnet add package Npgsql.EntityFrameworkCore.PostgreSQL
dotnet add package Microsoft.EntityFrameworkCore.Design
```

Paquete	Para qué sirve
Npgsql.EntityFrameworkCore.PostgreSQL	Driver que permite a EF Core hablar con PostgreSQL/Supabase
Microsoft.EntityFrameworkCore.Design	Necesario para los comandos de migraciones (dotnet ef ...)

6. Crear las carpetas del patrón

```
mkdir Controllers Services Repositories Models Data
```

7. Configuración — **appsettings.json**

Reemplaza el archivo con:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Host=db.tu-
proyecto.supabase.co;Port=5432;Database=postgres;Username=postgres;Password=tu_pas
sword"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

⚠ **Reemplaza** **db.tu-proyecto.supabase.co** y **tu_password** por los valores reales. Si olvidas esto, al correr las migraciones verás **Host desconocido**.

8. Capa MODEL — Models/Pago.cs

El modelo es la **foto de una fila** de la tabla `pagos`.

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace PortalPagos.Models
{
    [Table("pagos")]
    public class Pago
    {
        [Key]
        public Guid Id { get; set; } = Guid.NewGuid();

        [Required, Column("user_id")]
        public Guid UserId { get; set; }

        [Required]
        public string Servicio { get; set; } = string.Empty;

        [Column("numero_contrato")]
        public string NumeroContrato { get; set; } = string.Empty;

        [Column(TypeName = "decimal(10,2)")]
        public decimal Monto { get; set; }

        public string Estado { get; set; } = "completado";

        public DateTime Fecha { get; set; } = DateTime.UtcNow;
    }
}
```

Observa:

- `[Table("pagos")]` mapea la clase a la tabla `pagos` en la BD.
- `[Column("user_id")]` mapea `UserId` (C# PascalCase) a `user_id` (SQL snake_case).
- El modelo **NO** valida reglas de negocio (ej. qué servicios son válidos). Eso vive en el Service.

9. Capa DATA — Data/AppDbContext.cs

Es el "puente" entre tus clases y la BD. Cada `DbSet<T>` representa una tabla.

```
using Microsoft.EntityFrameworkCore;
using PortalPagos.Models;

namespace PortalPagos.Data
{
    public class AppDbContext : DbContext
```

```
{
    public AppDbContext(DbContextOptions<AppDbContext> options) :
base(options) { }

    public DbSet<Pago> Pagos { get; set; }
}
}
```

10. Capa REPOSITORY — Solo habla con la BD

10.1. Interfaz — `Repositories/IPagoRepository.cs`

```
using PortalPagos.Models;

namespace PortalPagos.Repositories
{
    public interface IPagoRepository
    {
        Task<List<Pago>> ObtenerPorUsuarioAsync(Guid userId);
        Task<Pago?> ObtenerPorIdAsync(Guid id);
        Task<Pago> CrearAsync(Pago pago);
        Task<Pago> ActualizarAsync(Pago pago);
        Task<bool> EliminarAsync(Guid id);
        Task<decimal> ObtenerTotalPagadoAsync(Guid userId);
    }
}
```

10.2. Implementación — `Repositories/PagoRepository.cs`

```
using Microsoft.EntityFrameworkCore;
using PortalPagos.Data;
using PortalPagos.Models;

namespace PortalPagos.Repositories
{
    public class PagoRepository : IPagoRepository
    {
        private readonly AppDbContext _context;

        public PagoRepository(AppDbContext context)
        {
            _context = context;
        }

        public async Task<List<Pago>> ObtenerPorUsuarioAsync(Guid userId)
        {
            return await _context.Pagos
                .Where(p => p.UserId == userId)
                .ToListAsync();
        }
    }
}
```

```
        .OrderByDescending(p => p.Fecha)
        .Take(20)
        .ToListAsync();
    }

    public async Task<Pago?> ObtenerPorIdAsync(Guid id)
    {
        return await _context.Pagos.FindAsync(id);
    }

    public async Task<Pago> CrearAsync(Pago pago)
    {
        _context.Pagos.Add(pago);
        await _context.SaveChangesAsync();
        return pago;
    }

    public async Task<Pago> ActualizarAsync(Pago pago)
    {
        _context.Pagos.Update(pago);
        await _context.SaveChangesAsync();
        return pago;
    }

    public async Task<bool> EliminarAsync(Guid id)
    {
        var pago = await _context.Pagos.FindAsync(id);
        if (pago == null) return false;

        _context.Pagos.Remove(pago);
        await _context.SaveChangesAsync();
        return true;
    }

    public async Task<decimal> ObtenerTotalPagadoAsync(Guid userId)
    {
        return await _context.Pagos
            .Where(p => p.UserId == userId)
            .SumAsync(p => p.Monto);
    }
}
}
```

Por qué una interfaz + una implementación: la inyección de dependencias (Paso 13) pide interfaces, no clases concretas. Así podrías mañana cambiar a un `PagoRepositoryMock` en tests sin tocar el Service.

11. Capa SERVICE — Reglas de negocio

11.1. Interfaz — `Services/IPagoService.cs`

```
using PortalPagos.Models;

namespace PortalPagos.Services
{
    public interface IPagoService
    {
        Task<List<Pago>> ObtenerPagosByUserAsync(Guid userId);
        Task<Pago?> ObtenerPagoPorIdAsync(Guid id);
        Task<Pago> RegistrarPagoAsync(Pago pago);
        Task<Pago> ActualizarPagoAsync(Guid id, Pago pago);
        Task<bool> EliminarPagoAsync(Guid id);
        Task<decimal> ObtenerTotalPagadoAsync(Guid userId);
    }
}
```

11.2. Implementación — `Services/PagoService.cs`

```
using PortalPagos.Models;
using PortalPagos.Repositories;

namespace PortalPagos.Services
{
    public class PagoService : IPagoService
    {
        private readonly IPagoRepository _pagoRepo;

        public PagoService(IPagoRepository pagoRepo)
        {
            _pagoRepo = pagoRepo;
        }

        public async Task<List<Pago>> ObtenerPagosByUserAsync(Guid userId)
        {
            return await _pagoRepo.ObtenerPorUsuarioAsync(userId);
        }

        public async Task<Pago?> ObtenerPagoPorIdAsync(Guid id)
        {
            return await _pagoRepo.ObtenerPorIdAsync(id);
        }

        public async Task<Pago> RegistrarPagoAsync(Pago pago)
        {
            var serviciosValidos = new[] { "agua", "electricidad", "streaming",
            "internet", "gas" };
            if (!serviciosValidos.Contains(pago.Servicio))
                throw new Exception("Servicio no valido");

            if (pago.Monto <= 0 || pago.Monto > 5000)
                throw new Exception("Monto fuera de rango (1 - 5000)");
        }
    }
}
```

```

        return await _pagoRepo.CrearAsync(pago);
    }

    public async Task<Pago> ActualizarPagoAsync(Guid id, Pago pago)
    {
        var existente = await _pagoRepo.ObtenerPorIdAsync(id);
        if (existente == null)
            throw new Exception("Pago no encontrado");

        var serviciosValidos = new[] { "agua", "electricidad", "streaming",
"internet", "gas" };
        if (!serviciosValidos.Contains(pago.Servicio))
            throw new Exception("Servicio no valido");

        if (pago.Monto <= 0 || pago.Monto > 5000)
            throw new Exception("Monto fuera de rango (1 - 5000)");

        existente.Servicio = pago.Servicio;
        existente.NumeroContrato = pago.NumeroContrato;
        existente.Monto = pago.Monto;
        existente.Estado = pago.Estado;

        return await _pagoRepo.ActualizarAsync(existente);
    }

    public async Task<bool> EliminarPagoAsync(Guid id)
    {
        return await _pagoRepo.EliminarAsync(id);
    }

    public async Task<decimal> ObtenerTotalPagadoAsync(Guid userId)
    {
        return await _pagoRepo.ObtenerTotalPagadoAsync(userId);
    }
}

```

Reglas de negocio vividas aquí:

- Solo acepta estos 5 servicios: **agua**, **electricidad**, **streaming**, **internet**, **gas**.
- El monto debe estar entre 1 y 5000.
- En el **PUT** se preservan **UserId** y **Fecha** originales (solo cambias campos editables).

Si mañana piden aceptar **"telefono"**, **solo tocas este archivo**. El Controller, Repository y Model no se enteran.

12. Capa CONTROLLER — **Controllers/PagosController.cs**


```
using Microsoft.AspNetCore.Mvc;
using PortalPagos.Models;
using PortalPagos.Services;

namespace PortalPagos.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class PagosController : ControllerBase
    {
        private readonly IPagoService _pagoService;

        public PagosController(IPagoService pagoService)
        {
            _pagoService = pagoService;
        }

        // GET /api/pagos?userId=uuid
        [HttpGet]
        public async Task<IActionResult> GetPagos([FromQuery] Guid userId)
        {
            var pagos = await _pagoService.ObtenerPagosByUserAsync(userId);
            return Ok(new { success = true, data = pagos });
        }

        // GET /api/pagos/{id}
        [HttpGet("{id:guid}")]
        public async Task<IActionResult> GetPagoPorId(Guid id)
        {
            var pago = await _pagoService.ObtenerPagoPorIdAsync(id);
            if (pago == null)
                return NotFound(new { success = false, message = "Pago no encontrado" });

            return Ok(new { success = true, data = pago });
        }

        // POST /api/pagos
        [HttpPost]
        public async Task<IActionResult> RegistrarPago([FromBody] Pago pago)
        {
            try
            {
                var resultado = await _pagoService.RegistrarPagoAsync(pago);
                return Created("", new { success = true, data = resultado });
            }
            catch (Exception ex)
            {
                return BadRequest(new { success = false, message = ex.Message });
            }
        }
    }
}
```

```

// PUT /api/pagos/{id}
[HttpPut("{id:guid}")]
public async Task<IActionResult> ActualizarPago(Guid id, [FromBody] Pago
pago)
{
    try
    {
        var resultado = await _pagoService.ActualizarPagoAsync(id, pago);
        return Ok(new { success = true, data = resultado });
    }
    catch (Exception ex)
    {
        return BadRequest(new { success = false, message = ex.Message });
    }
}

// DELETE /api/pagos/{id}
[HttpDelete("{id:guid}")]
public async Task<IActionResult> EliminarPago(Guid id)
{
    var eliminado = await _pagoService.EliminarPagoAsync(id);
    if (!eliminado)
        return NotFound(new { success = false, message = "Pago no
encontrado" });

    return Ok(new { success = true, message = "Pago eliminado" });
}

// GET /api/pagos/total?userId=uuid
[HttpGet("total")]
public async Task<IActionResult> GetTotalPagado([FromQuery] Guid userId)
{
    var total = await _pagoService.ObtenerTotalPagadoAsync(userId);
    return Ok(new { success = true, data = new { total_pagado = total }
});
}
}
}

```

Claves didácticas:

- `[Route("api/[controller]")]` — el `[controller]` se reemplaza por el nombre de la clase sin el sufijo: `PagosController` → `pagos` → ruta base `/api/pagos`.
- `[HttpGet("{id:guid}")]` — la restricción `:guid` obliga a que `id` sea un UUID válido. Si alguien manda `/api/pagos/abc`, ASP.NET responde 404 sin entrar al método.
- `[FromQuery]` lee `?userId=...`; `[FromBody]` lee el JSON del body.
- El Controller **solo** traduce HTTP ↔ Service. No tiene **if** de negocio.

13. Program.cs — Inyección de dependencias

```
using Microsoft.EntityFrameworkCore;
using PortalPagos.Data;
using PortalPagos.Repositories;
using PortalPagos.Services;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseNpgsql(builder.Configuration.GetConnectionString("DefaultConnection")))
    ;

builder.Services.AddScoped<IPagoRepository, PagoRepository>();
builder.Services.AddScoped<IPagoService, PagoService>();

builder.Services.AddControllers();
builder.Services.AddCors(opt =>
    opt.AddDefaultPolicy(p =>
        p.AllowAnyOrigin().AllowAnyMethod().AllowAnyHeader()));

var app = builder.Build();
app.UseCors();
app.MapControllers();
app.Run();
```

¿Qué es "inyección de dependencias"? En vez de que cada clase haga `new PagoRepository()`, le pides en el constructor `IPagoRepository`. ASP.NET se encarga de crear la instancia. Las líneas `AddScoped<A, B>()` dicen: "cuando alguien pida `A`, dale un `B`".

`AddScoped` = una instancia nueva por cada request HTTP.

14. Crear la tabla en Supabase con migraciones

14.1. Instalar la herramienta de EF (una sola vez por computadora)

```
dotnet tool install --global dotnet-ef
```

14.2. Generar y aplicar la migración

```
dotnet ef migrations add CrearTablaPagos
dotnet ef database update
```

¿Qué pasa?

1. `migrations add` lee tu `AppDbContext` y tus modelos, y genera un archivo C# en `Migrations/` con el SQL necesario.
2. `database update` ejecuta ese SQL en Supabase y registra la migración en `__EFMigrationsHistory`.

Después de esto verás la tabla `pagos` en Supabase.

15. Ejecutar la API

```
dotnet run
```

Vas a ver en la consola:

```
Now listening on: http://localhost:5095
```

El puerto `5095` (u otro) sale de `Properties/launchSettings.json`. **No es siempre 5000** aunque la documentación genérica así lo diga — siempre mira la consola.

16. Endpoints disponibles

Base URL: `http://localhost:5095/api/pagos`

#	Método	Ruta	Descripción
1	POST	<code>/api/pagos</code>	Registrar un pago
2	GET	<code>/api/pagos?userId={uuid}</code>	Listar los 20 últimos pagos de un usuario
3	GET	<code>/api/pagos/{id}</code>	Consultar un pago por ID
4	PUT	<code>/api/pagos/{id}</code>	Actualizar un pago
5	DELETE	<code>/api/pagos/{id}</code>	Eliminar un pago
6	GET	<code>/api/pagos/total?userId={uuid}</code>	Suma total pagada por un usuario

16.1. POST — Registrar

```
POST http://localhost:5095/api/pagos
Content-Type: application/json
```

```
{
  "userId": "f03363d1-c24d-463b-a654-4c38dd590007",
  "servicio": "internet",
  "numeroContrato": "123456",
```

```
"monto": 85.00
}
```

16.2. GET por ID

```
GET http://localhost:5095/api/pagos/c2527c83-cb5c-418f-ac20-bdf8746c32e0
```

16.3. PUT — Actualizar

```
PUT http://localhost:5095/api/pagos/c2527c83-cb5c-418f-ac20-bdf8746c32e0
```

```
{
  "userId": "f03363d1-c24d-463b-a654-4c38dd590007",
  "servicio": "electricidad",
  "numeroContrato": "999888",
  "monto": 250.50,
  "estado": "completado"
}
```

16.4. DELETE

```
DELETE http://localhost:5095/api/pagos/c2527c83-cb5c-418f-ac20-bdf8746c32e0
```

17. Flujo sugerido de prueba manual (Postman)

1. **POST** → crea un pago y copia el **id** de la respuesta.
2. **GET lista** → verifica que aparece.
3. **GET por ID** → con el **id** copiado.
4. **PUT** → cámbiale servicio y monto.
5. **GET por ID** → confirma los cambios.
6. **GET total** → suma de todos los pagos del usuario.
7. **DELETE** → bórralo.
8. **GET por ID** → ahora responde 404.

Extras para validar las reglas de negocio (todos deben dar **400 Bad Request**):

- POST con **"servicio": "cable"** → *"Servicio no valido"* (ya no está en la lista).
 - POST con **"monto": 6000** → *"Monto fuera de rango (1 - 5000)"*.
 - POST con **"monto": 0** → *"Monto fuera de rango (1 - 5000)"*.
-

18. Problemas comunes y soluciones

18.1. Host desconocido al correr migraciones

Causa: el `Host=` del connection string todavía dice `tu-proyecto.supabase.co` (el placeholder).

Solución: edita `appsettings.json` con tu host real de Supabase.

18.2. 42P07: relation "pagos" already exists

Causa: la tabla existe en Supabase (quizá creada antes manualmente) pero no está registrada en `__EFMigrationsHistory`.

Solución A (si no hay datos importantes): en el SQL Editor de Supabase:

```
DROP TABLE IF EXISTS pagos;  
DROP TABLE IF EXISTS "__EFMigrationsHistory";
```

Luego `dotnet ef database update` otra vez.

Solución B (si quieres preservar los datos): marca la migración como aplicada sin ejecutarla:

```
INSERT INTO "__EFMigrationsHistory" ("MigrationId", "ProductVersion")  
VALUES ('20260424165718_CrearTablaPagos', '8.0.0');
```

(usa el nombre exacto de migración de tu carpeta `Migrations/`).

18.3. ECONNREFUSED 127.0.0.1:5000 en Postman

Causa: probaste en puerto 5000 pero tu servidor usa 5095 (u otro).

Solución: mira la consola donde corre `dotnet run`, copia la URL `Now listening on:` `http://localhost:XXXX` y úsala en Postman.

18.4. Cambié el código pero Postman sigue igual

Causa: ASP.NET Core carga código compilado al arrancar. Los cambios no se aplican en caliente.

Solución: `Ctrl+C` en la terminal y `dotnet run` de nuevo. O usa **hot reload**:

```
dotnet watch run
```

Esto recarga automáticamente cuando editas archivos `.cs`.

18.5. GET por ID responde 404 con `{"success":false,"message":"Pago no encontrado"}`

Causa: el GUID que usaste no existe en la BD.

Solución: haz primero un `GET /api/pagos?userId=...`, copia un `id` real y usa ese.

19. Comparación mental (por si vienes de Laravel/Node)

Concepto	Laravel	Node/Express	ASP.NET Core
Crear proyecto	<code>composer create-project laravel/laravel</code>	<code>npm init</code>	<code>dotnet new webapi -n X</code>
Instalar paquete	<code>composer require X</code>	<code>npm install X</code>	<code>dotnet add package X</code>
Correr	<code>php artisan serve</code>	<code>node server.js</code>	<code>dotnet run</code>
ORM	Eloquent	Prisma / Sequelize	Entity Framework Core
Migraciones	<code>php artisan migrate</code>	<code>prisma migrate</code>	<code>dotnet ef database update</code>
Inyección de dep.	Service Container	Manual / Awilix	<code>Builder.Services.Add...</code>

20. Tabla resumen de capas

Capa	Archivo	Responsabilidad	Qué NO hace
ROUTER	Atributos <code>[HttpGet]</code> / <code>[HttpPost]</code> etc.	Mapear URLs a métodos	Lógica
CONTROLLER	<code>PagosController.cs</code>	Leer HTTP, devolver JSON, traducir errores	Reglas de negocio, SQL
SERVICE	<code>PagoService.cs</code>	Validar reglas, orquestar lógica	Tocar HTTP, tocar BD directo
REPOSITORY	<code>PagoRepository.cs</code>	Consultas EF/SQL	Validar reglas de negocio
MODEL	<code>Pago.cs</code>	Estructura de datos (mapeo a tabla)	Lógica, validaciones de negocio
DBCONTEXT	<code>AppDbContext.cs</code>	Puente EF ↔ BD	Nada más